Besides security, not having to recompile your application every time you want to tweak a value is one of the advantages of using configuration and settings. In the previous version of ASP.NET, changing a setting by editing web.config would cause your app to restart. This feature beats having to recompile, but waiting for the app to start up before it could serve requests was a bit of a drag.

In ASP.NET Core, you finally get the ability to edit a file and have the configuration of your application update automatically, without your having to recompile *or* restart. An often-cited scenario in which you might find this ability useful is when you're trying to debug an app you have in production. You typically configure logging to one of several levels:

- Error
- Warning
- Information
- Debug

Each of these settings is more verbose than the last, but it also provides more context. By default, you might configure your app to log only warning and error-level logs in production so that you don't generate too many superfluous log entries. Conversely, if you're trying to debug a problem, you want as much information as possible, so you may want to use the debug log level.

Being able to change configuration at runtime means that you can easily switch on extra logs when you encounter a problem and switch them back afterward by editing your appsettings.json file.

NOTE Reloading is generally available only for file-based configuration providers, such as the JSON provider, as opposed to the environment variable provider, for example.

You can enable the reloading of configuration files when you add any of the file-based providers to your `ConfigurationManager`. The `Add*File`